

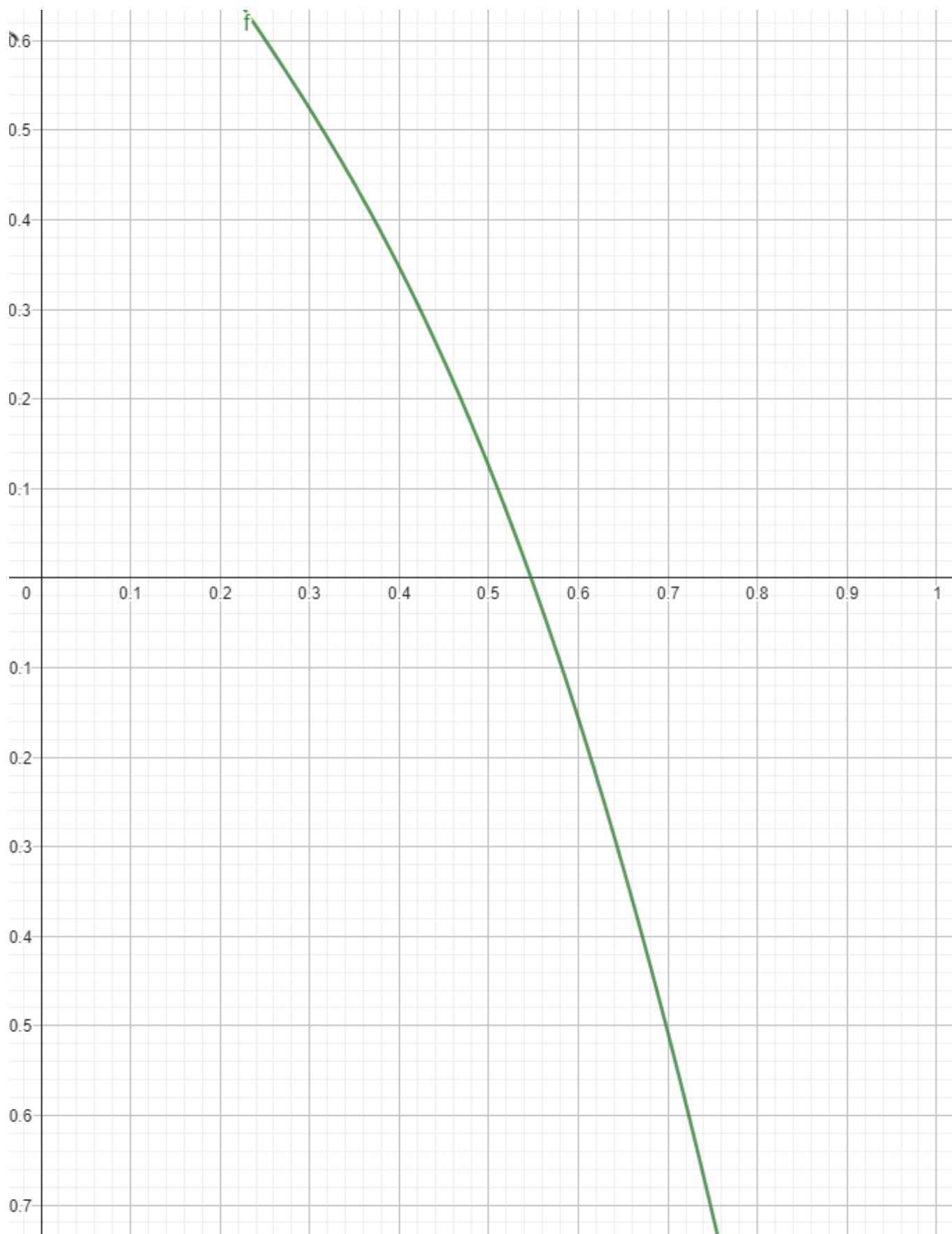
Выполнил: Радионов Никита Дмитриевич БПИ236

Вариант 32

В моём варианте необходимо методом половинного деления найти корень уравнения $2x^4 - 6x^3 + 3x^2 - 2x + 1 = 0$ на интервале $[0; 1]$ с заданной пользователем точностью, которая может быть в пределах от 0.001 до 0.00000001 .

Я провел предварительное исследование и выяснил, что на интервале $[0; 1]$ есть единственный корень, поэтому заданный по условию интервал корректен.

График функции на интервале $[0; 1]$



Для упрощения вычисления представим исходный полином в таком виде:

$$2x^4 - 6x^3 + 3x^2 - 2x + 1 = (((2x - 6)x + 3)x - 2)x + 1$$

Будем последовательно вычислять:

$$2x - 6$$

$$(2x - 6)x + 3$$

$$((2x - 6)x + 3)x - 2$$

$$(((2x - 6)x + 3)x - 2)x + 1$$

Основной алгоритм:

1. Считываем точность, учитывая что она должна быть в пределах от 0.001 до 0.00000001
2. Используем метод [половинного деления](#) для нахождения корня с заданной точностью и вывода его на экран

[Исходный код проекта](#)

Скриншот кода основной программы

The screenshot shows the RARS 1.6 IDE interface. The main window displays the assembly code for the file `compute_func_value.asm`. The code includes a macro library, defines a global `main` function, and contains a `main` routine that reads a double value from the keyboard, checks its accuracy, calls the `half_division_wrapper` function, prints the result, and exits. Comments in Russian explain each step: reading a double, checking accuracy, using the algorithm, and printing the answer. The status bar at the bottom indicates 'Line: 10 Column: 8' and 'Show Line Numbers' is checked. Below the code editor, the 'Messages' and 'Run I/O' panels show the output of the program: the accuracy value 0.00001, the computed result 0.5472450256347656, and a message indicating the program finished running successfully with exit code 0. A 'Clear' button is located at the bottom left of the Messages panel.

```
1 .include "macrolib.asm"
2 .global main
3
4 .text
5 main:
6     read_double(f$1)           # читаем double с клавиатуры
7     check_accuracy(f$1)        # проверяем корректность входных данных
8     half_division_wrapper(f$1) # используем основной алгоритм
9     print_double(f$a0)         # печатаем ответ
10    exit()
11
```

Line: 10 Column: 8 ☒ Show Line Numbers

Messages Run I/O

```
0.00001
0.5472450256347656
-- program is finished running (0) --
```

Clear

Пройдемся по каждому требованию и покажем, что оно выполняется.

Требования на оценку 4-5:

№1

Приведено решение задачи на ассемблере. Ввод данных осуществляется с клавиатуры. Вывод данных осуществляется на дисплей.

Выполнение этого требования доказывает скриншот выше.

№2

В программе должны присутствовать комментарии, поясняющие выполняемые действия.

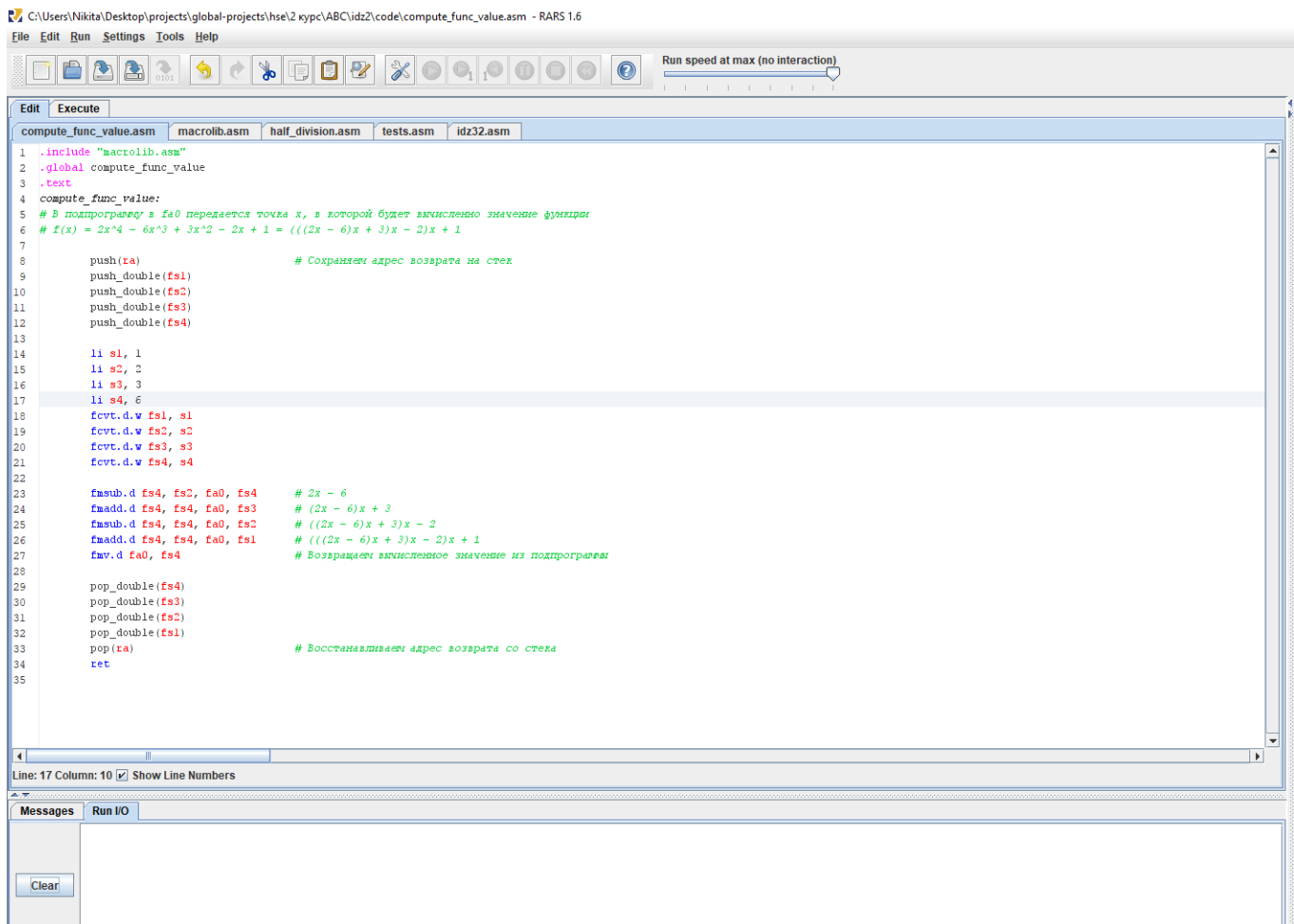
Как можно видеть со скриншота выше в основной программе комментарии присутствуют, как и в остальных частях многомодульного проекта.

№3

Допускается использование требуемых подпрограмм без параметров и локальных переменных.

Требование выполняется, поскольку подпрограммы есть и вынесены в отдельные модули.

Подпрограмма расчета значения функции вынесенная в отдельный модуль:



```
1 .include "macrolib.asm"
2 .global compute_func_value
3 .text
4 compute_func_value:
5 # В подпрограмме в f$0 передается точка x, в которой будет вычислено значение функции
6 # f(x) = 2x^4 - 6x^3 + 3x^2 - 2x + 1 = (((2x - 6)x + 3)x - 2)x + 1
7
8     push($a0)                # Сохраним адрес возврата на стек
9     push_double($f$1)
10    push_double($f$2)
11    push_double($f$3)
12    push_double($f$4)
13
14    li $t1, 1
15    li $t2, 2
16    li $t3, 3
17    li $t4, 6
18    fcvt.d.w $f$1, $t1
19    fcvt.d.w $f$2, $t2
20    fcvt.d.w $f$3, $t3
21    fcvt.d.w $f$4, $t4
22
23    fmsub.d $f$4, $f$2, $f$4    # 2x - 6
24    fmadd.d $f$4, $f$4, $f$3    # (2x - 6)x + 3
25    fmsub.d $f$4, $f$4, $f$2    # ((2x - 6)x + 3)x - 2
26    fmadd.d $f$4, $f$4, $f$1    # (((2x - 6)x + 3)x - 2)x + 1
27    fmv.d $f$0, $f$4          # Возвращаем вычисленное значение из подпрограммы
28
29    pop_double($f$4)
30    pop_double($f$3)
31    pop_double($f$2)
32    pop_double($f$1)
33    pop($a0)                  # Восстанавливаем адрес возврата со стека
34    ret
35
```

Подпрограмма алгоритма половинного деления вынесенная в отдельный модуль:

C:\Users\Nikita\Desktop\projects\global-projects\hse\2 kypc\ABC\ldz\code\half_division.asm - RARS 1.6

File Edit Run Settings Tools Help

Run speed at max (no interaction)

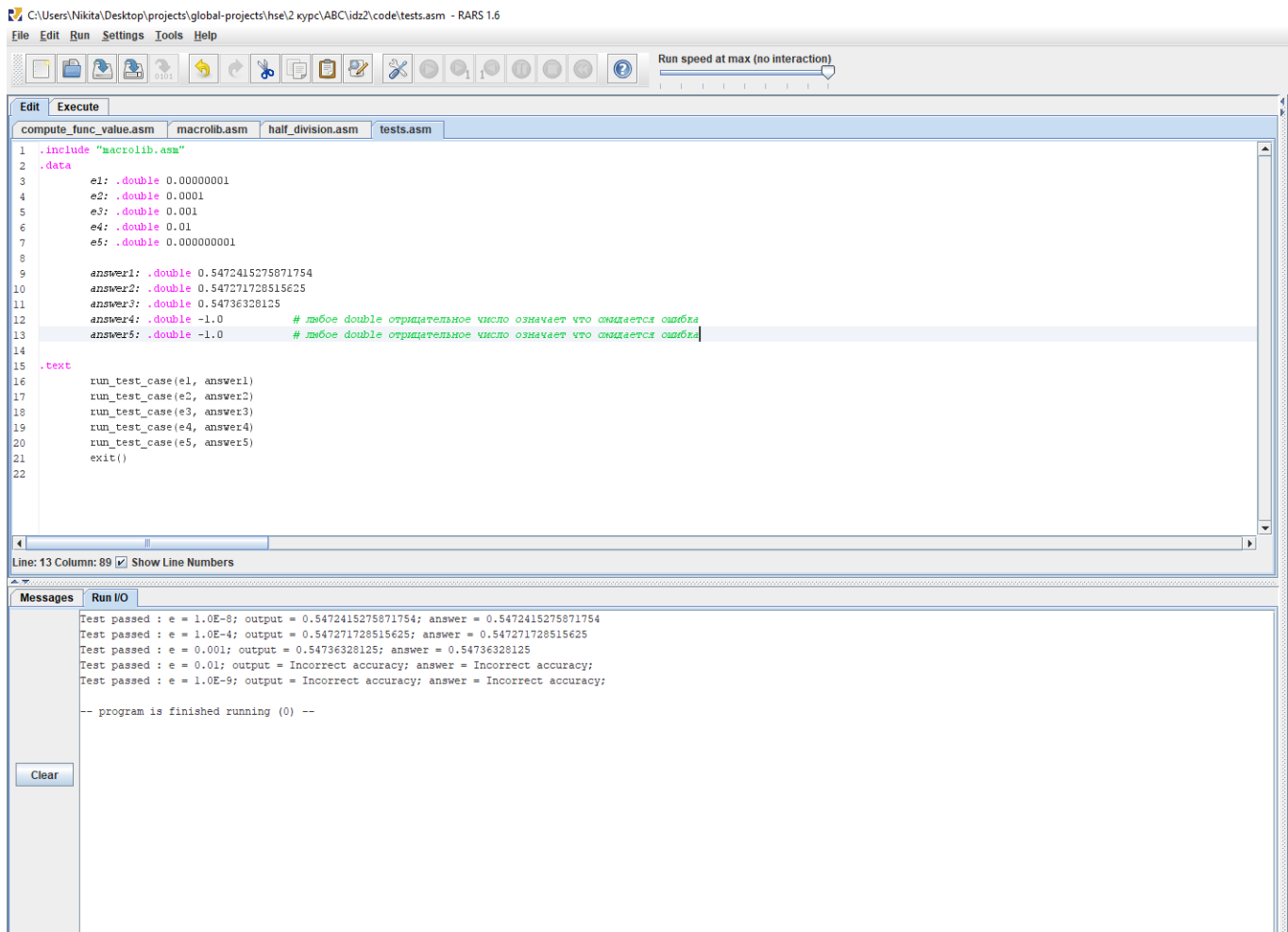
compute_func_value.asm macrolib.asm half_division.asm tests.asm idz32.asm

```
1 .include "macrolib.asm"
2 .global half_division
3 .text
4 half_division:
5 # В подпрограмме в fa0 передается значение точности e.
6 push(ra) # Сохраняем адрес возврата на стек
7 li t0, 0
8 li t1, 1
9 fmv.d ft6, fa0 # e
10 fcvt.d.w ft0, t0 # l = 0.0
11 fcvt.d.w ft1, t1 # l = 1.0
12
13 while_division:
14 fsub.d ft2, ft1, ft0 # x - 1
15 fgt.d t3, ft2, ft6 # x - 1 > e ?
16 beqz t3, end_while_division
17 fadd.d ft2, ft1, ft0 # x + 1
18 li t4, 2
19 fcvt.d.w ft4, t4
20 fdiv.d ft4, ft2, ft4 # c = (x + 1) / 2
21 compute_func_value_wrapper(ft4) # f(c)
22 fmv.d ft3, fa0
23 compute_func_value_wrapper(ft0) # f(l)
24 fmv.d ft2, fa0
25 fmul.d ft3, ft2, ft3 # f(l) * f(c)
26 li t2, 0
27 fcvt.d.w ft2, t2
28 fgt.d t3, ft2, ft3 # f(l) * f(c) < 0
29 beqz t3, false_comparision
30 fmv.d ft1, ft4 # x = c
31 j end_comparision
32 false_comparision:
33 fmv.d ft0, ft4 # l = c
34 end_comparision:
35 j while_division
36 end_while_division:
37 fadd.d ft2, ft1, ft0 # x + 1
38 li t4, 2
39 fcvt.d.w ft4, t4
40 fdiv.d ft3, ft2, ft4 # c = (x + 1) / 2
41 fmv.d fa0, ft3 # возвращаем c
42 pop(ra) # Восстанавливаем адрес возврата со стека
43 ret
44
```

Line: 21 Column: 40 ☒ Show Line Numbers

№4

В отчете должно быть представлено полное тестовое покрытие. Приведены результаты тестовых прогонов. Например, с использованием скриншотов.



Требования на оценку 6-7:

№5

В программе необходимо использовать подпрограммы с передачей аргументов через параметры, что должно обеспечивать их повторное использование с различными входными аргументами, включая, например, применение в других программах. При нехватке регистров, используемых для передачи параметров, оставшиеся пара метры передавать через стек.

Требование соблюдено.

Как видно из скриншотов пункта **№3** аргументы передаются подпрограммам в через регистры **fa****

№6

Внутри подпрограмм необходимо использовать локальные переменные или свободные регистры. То есть, подпрограмма должна быть полностью отделена от вызывающего ее кода. При нехватке временных регистров обеспечить сохранение данных на стеке в соответствии с соглашениями, принятыми для процессора.

Требование соблюдено.

Как видно из скриншотов пункта №3 локальные переменные находятся в регистрах **ft****

№7

В местах вызова функции добавить комментарии, описывающие передачу фактических параметров и местоположение возвращаемого результата. При этом необходимо отметить, какая переменная или результат какого выражения соответствует тому или иному фактическому параметру.

Требование выполняется, поскольку в месте вызова подпрограмм - в автономной библиотеке макроподстановок присутствуют комментарии по подготовке аргументов для их передачи в подпрограммы.

```
57
58 # Макро для вызова подпрограммы основного алгоритма:
59 .macro half_division_wrapper(%e)
60     fmv.d fa0, %e                # передаем подпрограмме половинного деления значение точности
61     jal half_division            # вызывает подпрограмму основного алгоритма
62 .end_macro
63
64
65 # Макро для вызова подпрограммы расчета значения функции в точке:
66 .macro compute_func_value_wrapper(%x)
67     fmv.d fa0, %x                # передает подпрограмме расчета значения функции точку
68     jal compute_func_value       # вызывает подпрограмму расчета значения
69 .end_macro
```

№8

Информацию о проведенных изменениях отобразить в отчете на ряду с информацией, необходимой на предыдущую оценку.

Вся необходимая информация приведена.

Требования на оценку 8:

№9

Разработанные подпрограммы должны поддерживать многократное использование с различными наборами исходных данных, включая возможность обработки в качестве параметров различных исходных данных.

Вызов подпрограмм и передача параметров им осуществляется внутри макроподстановок и их параметры не фиксированы, что позволяет использовать подпрограммы с различными наборами исходных данных.

```

57
58 # Макро для вызова подпрограммы основного алгоритма:
59 .macro half_division_wrapper(%e)
60     fmv.d fa0, %e          # передаем подпрограмме половинного деления значение точности
61     jal half_division      # вызывает подпрограмму основного алгоритма
62 .end_macro
63
64
65 # Макро для вызова подпрограммы расчета значения функции в точке:
66 .macro compute_func_value_wrapper(%x)
67     fmv.d fa0, %x          # передаем подпрограмме расчета значения функции точку
68     jal compute_func_value # вызывает подпрограмму расчета значения
69 .end_macro
70

```

№10

Реализовать автоматизированное тестирование за счет создания дополнительной тестовой программы, осуществляющей прогон подпрограмм, осуществляющих вычисления для различных тестовых данных (вместо их ввода). Осуществить прогон тестов обеспечивающих покрытие различных ситуаций. В том случае, если исходные данные напрямую не прописаны, а точность в условии зафиксирована, использовать организацию вычислений с различной точностью.

```

C:\Users\Nikita\Desktop\projects\global-projects\hse\2 кыпс\ABC\ids2\code\tests.asm - RARS 1.6
File Edit Run Settings Tools Help
Run speed at max (no interaction)
compute_func_value.asm  macrolib.asm  half_division.asm  tests.asm
1 .include "macrolib.asm"
2 .data
3     e1: .double 0.00000001
4     e2: .double 0.0001
5     e3: .double 0.001
6     e4: .double 0.01
7     e5: .double 0.000000001
8
9     answer1: .double 0.5472415275871754
10    answer2: .double 0.547271728515625
11    answer3: .double 0.54736328125
12    answer4: .double -1.0          # любое double отрицательное число означает что ожидается ошибка
13    answer5: .double -1.0          # любое double отрицательное число означает что ожидается ошибка
14
15 .text
16     run_test_case(e1, answer1)
17     run_test_case(e2, answer2)
18     run_test_case(e3, answer3)
19     run_test_case(e4, answer4)
20     run_test_case(e5, answer5)
21     exit()
22
Line: 13 Column: 89 [x] Show Line Numbers
Messages  Run I/O
Test passed : e = 1.0E-8; output = 0.5472415275871754; answer = 0.5472415275871754
Test passed : e = 1.0E-4; output = 0.547271728515625; answer = 0.547271728515625
Test passed : e = 0.001; output = 0.54736328125; answer = 0.54736328125
Test passed : e = 0.01; output = Incorrect accuracy; answer = Incorrect accuracy;
Test passed : e = 1.0E-9; output = Incorrect accuracy; answer = Incorrect accuracy;
-- program is finished running (0) --
Clear

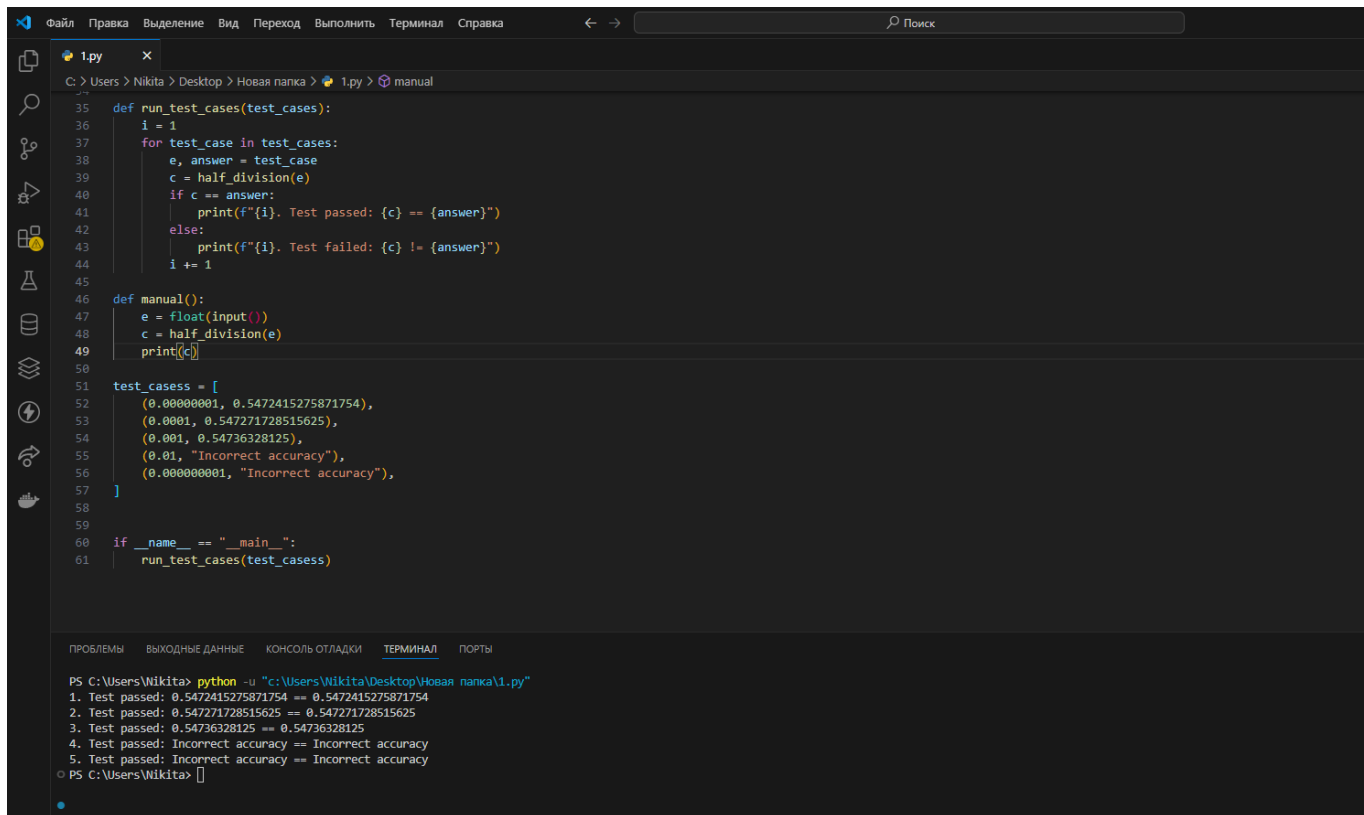
```

Как видно из скриншота работы программы, тесты автоматические.

№11

Для дополнительной проверки корректности вычислений осуществить аналогичные тестовые прогоны с использованием существующих библиотек и одного из языков программирования высокого уровня по выбору: C, C++, Python.

Я реализовал алгоритм на языке Python. Скриншот работы программы на тех же тестовых данных.



```
1.py
C:\Users\Nikita\Desktop\Новая папка > 1.py > manual

35 def run_test_cases(test_cases):
36     i = 1
37     for test_case in test_cases:
38         e, answer = test_case
39         c = half_division(e)
40         if c == answer:
41             print(f"{i}. Test passed: {c} == {answer}")
42         else:
43             print(f"{i}. Test failed: {c} != {answer}")
44         i += 1
45
46 def manual():
47     e = float(input())
48     c = half_division(e)
49     print(c)
50
51 test_cases = [
52     (0.00000001, 0.5472415275871754),
53     (0.0001, 0.547271728515625),
54     (0.001, 0.54736328125),
55     (0.01, "Incorrect accuracy"),
56     (0.000000001, "Incorrect accuracy"),
57 ]
58
59
60 if __name__ == "__main__":
61     run_test_cases(test_cases)

ПРОБЛЕМЫ ВХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ

PS C:\Users\Nikita> python -u "c:\Users\Nikita\Desktop\Новая папка\1.py"
1. Test passed: 0.5472415275871754 == 0.5472415275871754
2. Test passed: 0.547271728515625 == 0.547271728515625
3. Test passed: 0.54736328125 == 0.54736328125
4. Test passed: Incorrect accuracy == Incorrect accuracy
5. Test passed: Incorrect accuracy == Incorrect accuracy
PS C:\Users\Nikita>
```

№12

Добавить информацию о проведенных изменениях в отчет.

Вся необходимая информация приведена.

Требования на оценку 9:

№13

Добавить в программу использование макросов для реализации ввода и вывода данных. Добавить свои макросы, оберты, обертывающие подпрограммы обработки данных разработанные для решения основной задачи. Макросы должны поддерживать повторное использование с различными входными и выходными параметрами.

Требование соблюдается, поскольку в [библиотеке макроподстановок](#), есть макросы для реализации ввода и вывода данных, а также другие фрагменты программ.

Требования на оценку 10:

№14

Программа должна быть разбита на несколько единиц компиляции. При этом подпрограммы ввода–вывода должны составлять унифицированные модули, используемые повторно как в программе, осуществляющей ввод исходных данных, так и в программе, осуществляющей тестовое покрытие.

Требование выполняется, поскольку программа разбита на несколько единиц компиляции, а подпрограммы составляют унифицированные модули, которые используются в обоих случаях

№15

Макросы должны быть выделены в отдельную автономную библиотеку

Требование выполняется, поскольку все макроподстановки вынесены в отдельную библиотеку.

№16

Расширить отчет, дополнив его новыми данными.

Вся необходимая информация приведена.

[Исходный код проекта](#)